

SoftWall: Certified Multi-GPU Scheduling for Conditional Recovery in AI-RAN

ANONYMOUS AUTHOR(S)

AI-and-RAN schedulers share GPUs between deadline-sensitive radio processing and external AI. Existing resource shares and queue schedulers miss one class of work: until an optional neural receiver resolves, its transport block retains a conventional-recovery obligation that is mandatory under a future failure branch but absent from the current queue. Scheduling only visible work is unsafe; reserving every possible recovery strands capacity after neural success.

We present SoftWall, a certified multi-GPU scheduler for this conditional-work problem. It represents each unresolved obligation as recovery debt and jointly decides optional-receiver admission, conventional-recovery timing, and bounded external-AI leases. Every accepted decision preserves an executable all-fail schedule, and the same certificate follows the decision through IPC/P2P ownership, CUDA launch, physical completion, and radio commit. A provenance-qualified envelope predicts whether a state is safe-useful, safe-no-slack, mandatory-infeasible, or unqualified before execution.

Across 13,716 mandatory-feasible capacity points spanning one to four GPUs, conditional scheduling exposes capacity beyond static reservation at 3,600 (26.2%), while debt-blind scheduling has positive violation probability at 4,338 (31.6%). A frozen trace screen identifies 362 configurations with at least 5% scheduling headroom; a safe online greedy is within 5% of the fixed-placement oracle at 361, while flexible recovery placement has positive development-only upper-bound gain at 203. The original trace mode is a negative result: certificate-preserving recovery-first and SoftWall both deliver 349,387 timely tokens.

On two independent four-A100 nodes, physical boundary diagnostics produce 40/40 guard-and-deadline violations for debt-blind execution and 0/120 for certificate-preserving decisions. A separate qualified warm path executes 4,800 TensorRT NeuralRx requests, 1,312 shared-cuPHY recoveries, 1,077 Qwen units, and 4,800 radio commits without an observed deadline or invariant violation. The scheduler matches an exact checker on 16,023 states and all 180 prespecified physical boundary rounds; at 64 debts its p99 decision time is 1.494 ms. These results qualify synthetic modes rather than production HARQ; the 4.5 ms testMAC and external-channel exit gates remain explicit limitations.

CCS Concepts: • **Computer systems organization** → **Distributed architectures; Real-time systems; Networks** → *Network resources allocation*.

Additional Key Words and Phrases: AI-RAN, GPU scheduling, neural receiver, conditional recovery, MPS, feasibility certificate

ACM Reference Format:

Anonymous Author(s). 2027. SoftWall: Certified Multi-GPU Scheduling for Conditional Recovery in AI-RAN. In *Proceedings of ACM SIGMETRICS 2027 (SIGMETRICS '27)*. ACM, New York, NY, USA, 20 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

AI-and-RAN consolidates radio physical-layer processing and external inference on shared accelerators. The consolidation can reclaim idle GPU capacity, but it also couples a deadline-sensitive radio path to kernels whose arrivals and execution times follow a different service objective. Existing systems schedule GPU fractions, queued requests, accelerator placement, or bounded background units [1, 10, 11, 13]. Optional neural reception adds a scheduling state that none of these visible-work abstractions captures: future mandatory recovery.

Consider one uplink transport block (TB). A timely NeuralRx result may improve its decoding outcome, while failure, lateness, or lost evidence leaves conventional decoding mandatory. Before that outcome is known, the conventional path is absent from the execution queue but remains part of the radio contract. Several cells can hold this obligation at once, and several cuPHY instances

can send recovery to the same cuPHY lane. A scheduler that sees an idle GPU can therefore admit external AI into time that must remain available for correlated recovery.

Static reservation keeps every possible recovery slot until every neural outcome resolves. It is safe, but it strands the capacity released by timely neural success. A useful scheduler must decide three coupled questions: which speculative receivers to admit, where and when all unresolved conventional jobs could still execute, and which external-AI units can complete without invalidating that plan. Queue occupancy, MPS percentage, and independent per-instance schedules omit at least one of these decisions.

SoftWall is a certified event-driven scheduler built around *recovery debt*. Every unresolved optional path owns one debt, and each scheduling decision must leave a global executable all-fail calendar over the remaining debts. A success deletes one debt; failure, lateness, or missing evidence retains it. The scheduler may retime recovery and lease external AI only when an independent verifier accepts the joint schedule. MPS supplies concurrency, while the verified schedule supplies the radio contract.

A mathematical certificate alone is insufficient. In our implementation, a valid calendar became stale before kernel launch, simultaneous outcomes produced order-dependent intermediate states, a fixed lease omitted launch control, and reply loss left ownership ambiguous. SoftWall carries the same certificate through CUDA, IPC/P2P, and distributed-control state using bounded revalidation, atomic outcome batches, physical latest-start enforcement, generation-safe ownership, and fence-confirmed retirement. These mechanisms are established individually. Their combination refines an AI-RAN recovery invariant from admission to physical completion.

The paper makes four contributions:

- (1) **Conditional-recovery scheduling model.** We formalize unresolved same-TB recovery as debt and quantify safe AI capacity across cells, outcome correlation, deadlines, recovery cost, cuPHY instances, and one to four GPUs. Closed-form homogeneous capacity matches exhaustive checking.
- (2) **Certified multi-GPU scheduler.** An event-driven candidate scheduler jointly retimes recovery and leases external AI; an independent verifier accepts only schedules that preserve the all-fail radio branch. Seven measured counterexamples refine that decision through transport, worker epoch, CUDA fence, and single-commit state.
- (3) **Predictive feasibility envelope.** We classify each qualified state as qualified-safe-useful (QSU), qualified-safe-no-slack (QSN), mandatory-infeasible (MI), or unqualified (UQ) before physical execution and validate unopened physical boundaries.
- (4) **Scheduling evaluation with bounded claims.** Model, trace, exact-oracle, and two-node physical experiments separate safe capacity, decision quality, enforcement, and deployment scope. We retain the zero-gain trace result and do not claim an executed placement optimizer.

Claim boundary. The guarantee is conditional on the qualified bounds and provenance of the evaluated mode. SoftWall does not claim that MPS provides hard isolation, that finite-sample maxima are worst-case execution times (WCETs), or that the synthetic timing contract is a production HARQ deadline. Five lifecycle subsets are qualified or partially qualified and five remain UQ; the claim does not require or imply a 10/10 lifecycle matrix.

2 Background and Motivation

2.1 AI-and-RAN on a shared GPU

A shared AI-RAN node executes three scheduling classes: speculative NeuralRx, mandatory conventional recovery, and external AI. Figure 1 follows one uplink TB through one release period of

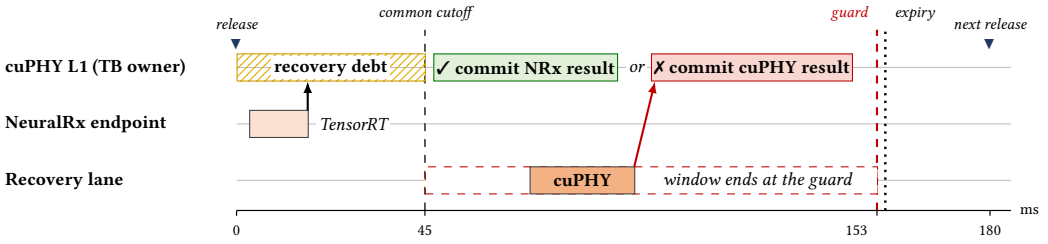


Fig. 1. One uplink TB in one 180 ms release period. Until the 45 ms common cutoff, the TB owes a conventional recovery that no GPU queue contains. A failed, late, or missing NeuralRx outcome requires one cuPHY recovery that ends before the guard.

the evaluated contract. Until the common cutoff shows a timely NeuralRx success, the TB owes a conventional recovery that no GPU queue contains. The scheduler must therefore track both the radio branch and the physical resource on which that branch may execute.

MPS Execution. MPS permits CUDA processes to execute concurrently and limits a client's active-thread percentage [7]. These controls choose how visible kernels share a GPU; they do not bound host scheduling, queueing, context lifecycle, allocation, or arbitrary kernel blocking. We therefore use MPS as the execution mechanism and make scheduling admission depend on a separate timing contract.

MIG-Off Reclamation. MIG provides spatial separation, while fixed partitions restrict reclaiming capacity released by a successful optional receiver. Our MIG-off setting keeps this capacity schedulable and uses qualified MPS modes for concurrency. A missing bound makes the mode UQ. Capacity is therefore reclaimed only within the placement and lifecycle for which service bounds were established.

2.2 Optional NeuralRx creates future mandatory work

An optional receiver is speculative radio execution. It can improve the TB outcome, but its dispatch does not remove the requirement to produce a conventional result after failure, lateness, or lost evidence. This asymmetry creates a job whose release depends on the future receiver outcome.

Same-TB Dependency. Before the NeuralRx outcome resolves, conventional decoding is conditional in the observed execution and mandatory in an allowed future branch. It therefore need not appear in the current GPU queue even though the scheduler must keep it executable. This gap between queued work and obligated work is the central scheduling problem.

Shared Recovery. Several cells can select NeuralRx together, and their failures can be correlated. When their conventional paths share one cuPHY lane, feasibility belongs to the union of their unresolved jobs. A schedule that is valid at each cuPHY instance can therefore be invalid after the instances' jobs are composed.

External-AI Admission. Qwen is a representative external workload that creates physical GPU contention; it is not an AI-RAN-domain workload. Its deadline and complete launch-to-fence service form a third job class. Admitting it before an unresolved radio outcome is useful only when the remaining conventional jobs still have a feasible placement.

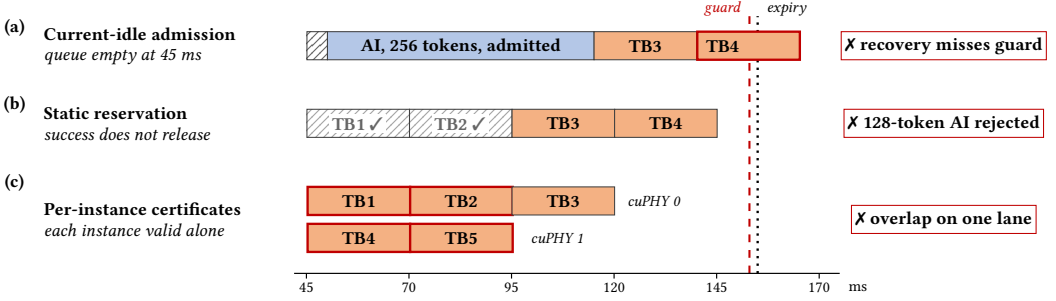


Fig. 2. Scheduling failures after two timely successes (TB1, TB2). (a) Current-idle admission pushes a retained recovery past the guard. (b) Static reservation rejects a 128-token AI unit that fits once the successful reservations are released. (c) Per-instance certificates claim one lane interval.

2.3 Threat model and qualification

SoftWall provides a conditional guarantee for an explicitly qualified mode. Qualification binds service bounds to the environment in which they were measured and defines the events the runtime must contain.

Mode Provenance. A mode fixes GPU topology, MPS settings, process lifecycle, worker epoch, memory residency, co-run class, software, and finite-sample whole-path bounds. Changing any element requires requalification. The synthetic release period and result expiry used in evaluation are harness parameters, not NR slot or production HARQ timing.

Contained Events. The runtime handles NeuralRx failure and lateness, stale and duplicate messages, bounded reply loss before or after a control-plane effect, missing AI replies after physical completion, and same-worker channel reconnection. It fails closed when terminal physical evidence is absent.

Excluded Events. Arbitrary GPU hangs, driver reset, unbounded host suspension, production-clock uncertainty, and unqualified workloads remain outside the claim. Mandatory offered load without an all-fail schedule is MI rather than a reason to weaken the safety check.

2.4 Missing scheduling state

Visible queues and static reservations fail in opposite directions. Figure 2 separates the three scheduling failures that SoftWall must avoid: admitting AI into apparently idle time needed by future recovery, reserving conditional work after it becomes unnecessary, and composing locally valid calendars on one shared lane. These failures define the state and scope of the scheduler.

Takeaway 1: current idleness is not schedulable slack. An unresolved NeuralRx leaves mandatory work outside the current queue. A scheduler needs an explicit representation of that work before it can safely place external AI.

Takeaway 2: worst-case reservation is safe but incomplete. Timely NeuralRx success deletes recovery work and changes the feasible schedule. A useful scheduler must recompute the calendar at outcome events rather than hold every initial reservation until expiry.

Takeaway 3: feasibility follows the contended lane. Local admission cannot protect a shared cuPHY worker. The certificate must cover the global obligation set and update all affected recovery placements in one transaction.

These scheduling rules are still insufficient unless they survive physical execution. SoftWall uses bounded admission, list scheduling, generation checks, CUDA fences, IPC/P2P rings, and at-most-once ownership to refine one conditional-recovery invariant. Table 1 shows why each refinement is necessary; Figure 3 later maps the resulting transaction to the runtime. All measured results appear in Section 6.

Table 1. Counterexamples define the certificate-to-execution refinement chain.

Counterexample	Broken assumption	Required refinement
Local-safe/global-unsafe	Local certificates compose on a shared lane	Global obligation set and certificate
Dispatch tail	A certificate stays fresh until launch	Bounded revalidation and physical latest-start
Sequential cutoff replay	Simultaneous outcomes may apply in any order	Atomic outcome batch
Fixed AI lease	Execution time covers launch control	Control charge and AI blackout
Reply loss after effect	Retrying ambiguity is harmless	Generation quarantine and fail-closed admission
Two outstanding pre-pares	RPC at-most-once implies visible ownership	Single unlaunched-token invariant
Warm-bound reuse	Lifecycle does not affect service tails	Provenance-scoped qualification

From Local to Global. Recovery feasibility belongs to the contended physical lane. The certificate must cover the union of obligations before optional endpoint or external-AI admission.

From Decision to Launch. A certificate must remain current through control delay, outcome observation, and CUDA launch. A lease therefore includes launch control and carries an absolute latest-start enforced by the worker.

From Logical Completion to Physical Retirement. RPC success is not CUDA completion, and reply loss is not proof that no effect occurred. Generations, terminal markers, and physical fences bind ownership to retirement. Section 3 formalizes this invariant and maps it to executable components.

3 SoftWall Design

SoftWall turns conditional recovery into explicit scheduling state and preserves that state until GPU effects become physically unambiguous. This section first presents the complete scheduler, then expands the two transitions that make its decisions safe, and finally gives the event algorithm and capacity model used by the implementation.

3.1 Overall architecture

Figure 3 shows the scheduling loop on its runtime objects. Persistent MPS workers own GPU work, while one host coordinator owns the per-TB decision pages, the recovery-debt table, the recovery-lane certificate, and the verifier. The four numbered arrows mark the cross-layer steps of one epoch, and the shared lane runs retained recoveries in certificate order. The same control layer serves a single GPU, where CUDA IPC rings connect the workers, and several GPUs, where retained debts from different cuPHY instances share one recovery lane over NVLink P2P.

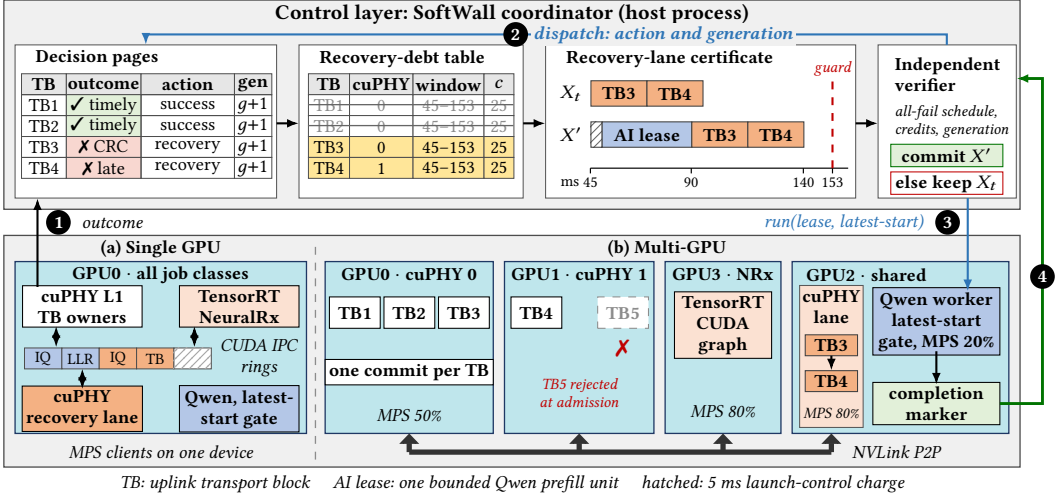


Fig. 3. SoftWall architecture. The control layer is identical for both placements. (1) Each owner publishes its NeuralRx outcome to a decision page. One common-cutoff batch then updates the debt table, the scheduler retimes retained debts around a candidate lease, and the independent verifier commits X' or keeps X_t . (2) The coordinator publishes each action with its generation and (3) sends the lease with an absolute latest-start. (4) A completion or non-launch marker retires the lease. The shared lane runs retained recoveries in certificate order. In (a), the processes on one GPU exchange IQ, LLR, and TB buffers through CUDA IPC rings. In (b), the evaluated multi-GPU placement uses NVLink P2P; cuPHY instance 0 owns TB1–TB3 and instance 1 owns TB4 and TB5.

Scheduling objects. The scheduler admits only information visible at the current event. A radio job and an external-AI job are

$$J_i = (r_i, d_i, h_i, c_i, K_i, q_i), \quad A_a = (u_a, v_a, \ell_a, b_a, g_a), \quad (1)$$

where r_i, d_i are release and expiry, h_i is the owning cuPHY instance, c_i is the conventional host-to-commit bound, K_i contains eligible NeuralRx endpoints, and q_i is optional radio value; $u_a, v_a, \ell_a, b_a, g_a$ are AI arrival, timely value, deadline, physical-completion bound, and recovery guard. Decisions cannot use future CRC, arrivals, realized service, or hidden channel labels.

Qualified modes. A number becomes scheduling authority only with matching provenance

$$\pi = (\text{GPU}, \text{placement}, \text{MPS}, \text{lifecycle}, \text{co-run}, \text{software}). \quad (2)$$

A warm local measurement therefore cannot certify a cold, restarted, or remote path. The scheduler marks a state UQ whenever a required bound lacks the active provenance, even if a sample happens to finish before its deadline.

Separation of proposal and authority. The candidate scheduler may use a fast list heuristic and may reject a feasible placement. It cannot create a false-safe execution because only the verifier can commit a candidate. With this separation, the evaluation compares decision quality with an exact oracle while safety remains independent of the heuristic.

3.2 Scheduling conditional recovery

An unresolved NeuralRx is a scheduling obligation even when no conventional kernel is queued. SoftWall records that obligation as recovery debt

$$R_i = (a_i(t), d_i - g_i, c_i, h_i), \quad (3)$$

where $a_i(t)$ is its earliest legal start. For unresolved set $U(t)$, an all-fail certificate assigns every R_i a non-overlapping interval $[s_i, s_i + c_i]$ on its physical recovery lane, after release and before $d_i - g_i$. Contending transport and AI blackout intervals belong to the same certificate.

All-Fail Dominance.

PROPOSITION 1 (ALL-FAIL DOMINANCE). *If a timely NeuralRx success only deletes its conventional recovery and does not increase any remaining job's demand, a feasible certificate for $U(t)$ is feasible for every failure subset $F \subseteq U(t)$.*

PROOF. Delete successful requests from the all-fail schedule. Release, expiry, capacity, and non-overlap constraints of every retained interval remain unchanged. $\square$

The reduction replaces $2^{|U|}$ outcome schedules with one executable witness. It does not apply when success creates new mandatory work or changes another service bound; that case needs a new mode.

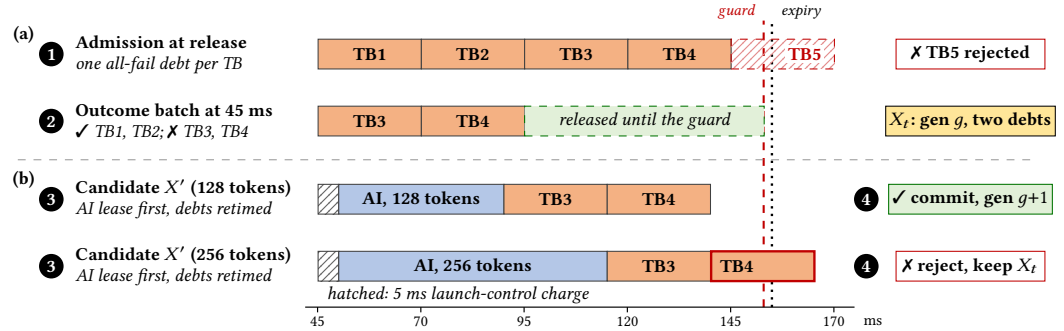


Fig. 4. One transaction on the shared lane. (a) Admission reserves one all-fail recovery per TB and rejects TB5; the 45 ms batch deletes two debts in one generation. (b) The verifier commits the 128-token AI lease and keeps X_t for the 256-token AI lease, whose last recovery would end after the guard. AI units are Qwen prefill requests; hatching marks the 5 ms launch-control charge.

Mandatory-first transition. Figure 4a reserves R_i before optional dispatch. Endpoint rejection therefore never removes the conventional path. All outcomes observed at one cutoff enter one transition: timely success deletes debt, while failure, lateness, and missing evidence retain it. Sequential replay is invalid because it can manufacture intermediate states that did not coexist.

Global atomic state. Certificate scope follows the contended lane: disjoint lanes compose by union, while cuPHY instances sharing a recovery lane require one global obligation set. The complete runtime state is

$$X_t = (S_t, Q_t, T_t, L_t, E_t, C_t, G_t), \quad (4)$$

where S is the recovery schedule, Q, T endpoint and transport credits, L AI leases, E fences and worker epochs, C radio commit state, and G multi-instance AI ownership. Figure 4b commits a

private X' only when

$$\begin{aligned} \text{VALID}(X') \equiv & \text{ExecutableRecovery}(S') \wedge \text{UniqueCredits}(Q', T') \\ & \wedge \text{TimelyAI}(L', E') \wedge \text{CurrentGenerations}(X'). \end{aligned} \quad (5)$$

Failure leaves X_t intact, eliminating partial calendar updates and orphan AI leases.

Safety Invariant.

SAFETY INVARIANT 1 (CERTIFICATE-TO-EXECUTION REFINEMENT). *For every admitted radio request, every allowed unresolved-outcome branch retains a collision-free conventional path completing by expiry. No endpoint, transport, AI, or output credit is reused before matching terminal physical evidence, and each request commits at most one radio result.*

3.3 Refining schedules to physical completion

A feasible host calendar is insufficient unless launch and ownership obey it. Figure 5 follows one AI lease from commit through the latest-start gate, CUDA execution, and the marker that retires it, together with the injected faults. The mechanism closes the interval between a scheduling decision and the CUDA effect it authorizes.

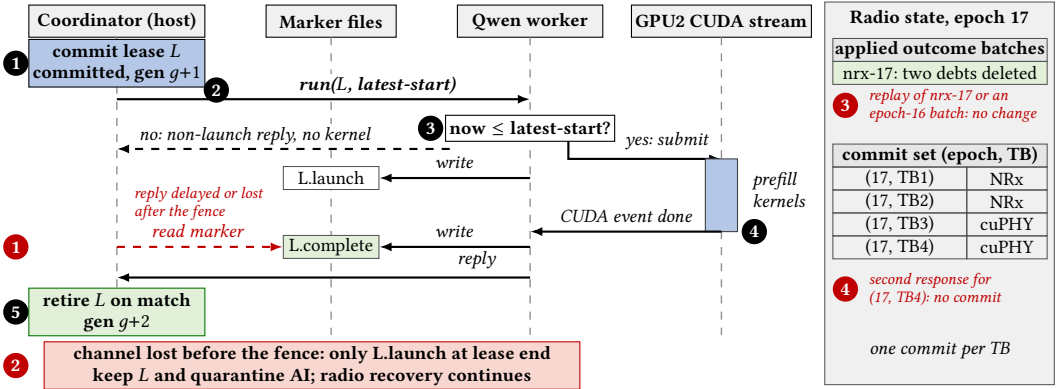


Fig. 5. Refinement of one AI lease. Black steps: commit with a latest-start (1), run request (2), launch gate and launch marker (3), CUDA event and completion marker (4), and retirement on a matching marker (5). Red faults: marker reconciliation after a late reply (1), AI quarantine after channel loss (2), and no-op outcome and response replays (3, 4).

Bounded launch. Prepare creates one unlaunched token carrying the calendar generation, worker epoch, ring and transport credit, and absolute latest-start. After that time, a worker emits a qualified non-launch marker and does not execute the stale schedule. The executor runs only the first non-preemptive action, observes its terminal event, and returns control to the scheduler.

Physical retirement. RPC completion is not GPU completion. Leases and credits therefore remain owned until a matching CUDA fence. A timeout without physical proof quarantines the generation and closes new optional admission. Stale, duplicate, and reply-lost messages cannot retire newer ownership, and a transport-copy fence cannot stand in for receiver or recovery completion.

Distributed ownership. One global AI queue may serve several cuPHY instances, but each instance may hold at most one unlaunched token in an ownership epoch. Commit transfers launch authority only after local and global certificates remain current. Same-worker reconnect carries the generation through bounded markers; full process replacement without durable reconciliation remains UQ.

Why IPC/P2P is part of scheduling. CUDA IPC and NVLink P2P avoid a host-staging copy, but their main design role is explicit ownership. Each ring slot has one generation, one physical completion state, and one scheduler-visible retirement event. Transport is therefore charged inside the qualified service path and is not treated as a zero-cost link between an abstract scheduler and cuPHY/NeuralRx.

3.4 Certified event-scheduling algorithm

SoftWall is an event-driven scheduling algorithm with a proof-carrying commit point. Algorithm 1 applies simultaneous outcomes, restores an all-fail calendar, and then searches currently visible AI candidates. It rejects optional work whenever provenance, recovery feasibility, or ownership is not provable.

Algorithm 1: CERTIFYEVENT($X_t, \mathcal{E}_t, \mathcal{A}_t, \pi$)

- (1) If any required bound disagrees with π , return UQ and disable new optional work.
- (2) Apply the event batch $\mathcal{E}_t$ atomically: reserve debt for new radio jobs; delete debt only for timely successes; retain debt for fail, late, or missing evidence.
- (3) Place every retained debt on its qualified physical lane. If no all-fail calendar exists, return MI and reject new NeuralRx and AI admission.
- (4) Visit only visible AI jobs in deterministic policy order. For each job, construct X' by jointly retiming recovery and adding one complete AI lease.
- (5) If the independent verifier accepts X' , atomically commit it, dispatch its first action, and return QSU. Otherwise discard X' without changing X_t .
- (6) If no AI candidate passes, commit the recovery-only calendar and return QSN. A matching terminal event retires ownership and invokes CERTIFYEVENT again.

Fig. 6. SoftWall's online scheduling algorithm. Candidate order controls efficiency; the verifier controls safety. Consequently, a heuristic can be false-conservative but cannot authorize a false-safe state.

Decision window. For m_h equal-length recoveries compacted at expiry d_h with guard g_h , let b_{eff} include synchronous control, AI service, and completion guard. AI-first execution is safe under the sufficient, mode-specific condition

$$T_{\text{dec},h} + b_{\text{eff}} \leq d_h - g_h - m_h c_h. \quad (6)$$

The right side is the first recovery start retained by the certificate. The inequality therefore charges decision and launch delay rather than assuming immediate dispatch.

Current Idleness.

PROPOSITION 2 (CURRENT IDLENESS IS INSUFFICIENT). *On one non-preemptive lane with common guard D , unresolved bounds $c_1, \dots, c_m$, and AI bound b_{eff} , suppose mandatory-only execution is feasible but*

$$t + b_{\text{eff}} + \sum_{i=1}^m c_i > D. \quad (7)$$

Then admitting AI from current idleness while ignoring unresolved recovery cannot guarantee the radio contract.

Table 2. The scheduler's predictive outcomes.

Class	Meaning
QSU	All-fail recovery is executable and at least one qualified AI unit is admissible.
QSN	All-fail recovery is executable, but the visible AI candidate cannot safely fit.
MI	The mandatory all-fail schedule is infeasible under known bounds.
UQ	A required bound or lifecycle provenance is absent or failed qualification.

PROOF. Choose the allowed realization in which AI and every recovery take their declared bounds. The final recovery completes after D by Equation 7; present idleness does not erase future mandatory demand. $\square$

Safe reclaimable capacity. The same invariant predicts the capacity that the algorithm may expose. Let W be the guard window, L the AI deadline, a one complete AI transaction, c one recovery bound, M_g potential static debts, and F_g realized debts on lane g . With $[x]_+ = \max(0, x)$,

$$K_{\text{static}} = \sum_g \left\lfloor \frac{[\min(L, W - M_g c)]_+}{a} \right\rfloor, \quad (8)$$

$$K_{\text{RF}} = \sum_g \left\lfloor \frac{[\min(L, W) - F_g c]_+}{a} \right\rfloor, \quad (9)$$

$$K_{\text{SoftWall}} = \sum_g \left\lfloor \frac{[\min(L, W - F_g c)]_+}{a} \right\rfloor. \quad (10)$$

Static holds all potential debt; recovery-first executes realized debt before AI; SoftWall may put AI first while retaining $F_g c$ before the guard. Multi-class BurstGPT batches use exact subset, lane, order, and recovery-insertion enumeration. Failure probabilities predict expected value, while the all-fail safety certificate is distribution-free.

The four outcomes keep scheduling failure distinct from missing authority. QSU and QSN preserve the same radio invariant; MI proves overload under known bounds; UQ refuses to reuse evidence from a different physical mode.

4 Implementation

The prototype binds the model to persistent CUDA workers and explicit control objects. Its purpose is to make certificate state observable and enforceable across process and GPU boundaries.

4.1 Runtime topology

The implementation runs with MIG disabled and MPS enabled. cuPHY instances, neural endpoints, conventional recovery, and external AI execute in separate persistent workers.

Radio and AI Workers. Local and remote TensorRT NeuralRx workers hold preallocated bindings and CUDA graphs. Persistent Aerial/cuPHY workers execute conventional recovery, while a Qwen2.5-1.5B worker executes bounded prefill classes. Multi-instance modes direct conventional work to one shared recovery GPU. In the evaluated placement (Figure 3b), GPU0 and GPU1 host the cuPHY instances that own the TBs with a 50% MPS thread limit, GPU3 hosts the NeuralRx endpoint, and GPU2 hosts the shared cuPHY lane and the Qwen worker, which runs as a low-priority MPS client with a 20% thread limit.

Transport. CUDA IPC rings connect processes on one GPU, and NVLink P2P rings connect GPUs. Each slot carries a request generation and physical completion state. Copy completion does not imply receiver or recovery completion; the owning worker publishes a separate terminal fence.

Controller. The controller stores recovery calendars, endpoint and transport credits, worker epochs, global AI ownership, and single-commit state. Unix-domain RPC carries control messages, while CUDA-resident data stays on IPC/P2P paths.

4.2 Certificate engine

The certificate engine separates schedule construction from admission authority. This separation contains candidate-scheduler bugs and makes each decision independently checkable.

Candidate Scheduler. The scheduler applies deterministic list orders to radio jobs and visible AI candidates. It emits a complete schedule, ownership delta, and mode fingerprint. Exact search is used only as a small-state oracle and is not an optimization contribution.

Independent Verifier. The verifier checks job identity, release, qualified service, expiry, lane capacity, overlap, AI blackout, credit uniqueness, and generation. The runtime cannot execute a candidate rejected by the verifier. This interface also supports retrospective comparison against exact feasibility.

Event Loop. The executor performs the first non-preemptive action in an accepted certificate, observes its terminal event, and replans. It never reorders physical work from an unverified host-side queue.

4.3 Measurement and mode separation

Instrumentation distinguishes logical decisions, host progress, GPU execution, and physical retirement. This distinction prevents a host timestamp from being reported as a kernel boundary.

Recorded Events. The runtime records release, admission, dispatch, worker observation, CUDA completion, commit return, request and payload identity, ring generation, worker epoch, and lease retirement. Nsight traces check GPU ordering for selected modes.

Python and Native Paths. The qualified C158-C162 path uses pyAerial with persistent workers. A later native C++ effort verifies input-layout and decoder parity only. It is not timing-qualified and its measurements are never mixed with the Python mode's service bounds.

Artifact Binding. Source hashes, protocols, seeds, node exclusions, analyzer versions, failures, and result hashes are preserved in manifests. Every empirical block in this paper maps to a machine-readable artifact and field set.

5 Evaluation Methodology

The evaluation follows the scheduler's claim chain. It first measures whether conditional scheduling adds safe capacity and how close its online decisions are to an oracle. It then tests prediction, physical enforcement, fault closure, and the boundaries beyond which the scheduler has no authority.

5.1 Questions and metrics

We organize the experiments around seven questions. Table 3 connects each question to the claim it can support.

Safety Metrics. We count radio expiry misses, declared-bound violations, guard crossings, invalid certificates, duplicate commits, stale generation acceptance, transport-credit reuse, and residual leases. Zero observed events is reported as a finite-sample result rather than a probability or WCET claim.

Table 3. Evaluation questions and their decision criteria.

RQ	Question	Decision criterion
1	What scheduling advantage does debt awareness provide?	More safe capacity than static reservation and no debt-blind false-safe admission.
2	How good are the online decisions?	Gap to exact/oracle schedules and sensitivity to each scheduling lever.
3	Can the scheduler predict and scale?	Exact, retrospective, and unopened-boundary decisions agree; construction stays bounded.
4	Does the accepted schedule survive physical execution?	Radio, recovery, AI, credit, and commit gates all pass.
5	Does refinement contain specified faults?	State audit and physical fault classes preserve invariants.
6	Why are MPS and lifecycle state part of the mode?	Cap-only and provenance-changing diagnostics expose violations.
7	Where does deployment authority end?	Production timing and channel gates retain failures.

Utility Metrics. External-AI utility is deadline-compliant token value and completed request count. Radio utility is correct TB outcome under the declared channel mode. Candidate-policy comparisons preserve the same radio decisions and observable inputs.

5.2 Testbed, workloads, and contract

Authoritative campaigns use two independent nodes, each with four A100-SXM4-40GB GPUs connected by NVLink. Facility identity is withheld for anonymous review. MIG is off and MPS is on.

Radio and External AI. Radio execution uses NVIDIA Aerial/cuPHY and TensorRT NeuralRx. Qwen2.5-1.5B prefill creates external GPU contention. BurstGPT supplies arrivals and token values from four non-overlapping public-trace windows [15]; Qwen is not labeled an AI-RAN-domain workload.

Qualified Synthetic Contract. The evaluated warm mode uses harness period/expiry P180/D155, with finite-sample whole-path bounds $B_{\text{NRx}} = 45$ ms, $B_{\text{conv}} = 25$ ms, $B_{\text{AI}}(\text{context64}) = 35$ ms, and a 2 ms guard. The period is not an NR slot, and the expiry is not a production HARQ deadline. Other AI contexts use their separately qualified work class.

Channel Modes. Synthetic channels drive system qualification. A separate Sionna CDL-D/E mode tests radio value under its own channel, radio, model, and precision contract. Aerial TDL-A and field IQ remain outside that supported mode.

5.3 Protocols and baselines

Each authoritative campaign freezes its decision rule before opening the outcome. Development and holdout runs use independent seeds and nodes where the gate requires them.

Experimental Discipline. Protocols fix source hash, workload, bounds, seeds, node assignment, exclusions, analyzer, and stop rule. Failed, negative, and UQ runs stay in the ledger. A model-predicted boundary is frozen before physical execution.

Safe Baselines. *Static global-safe* retains worst-case recovery reservation. *Certificate-preserving recovery-first* uses the same max-radio decisions, initial all-fail admission, arrivals, bounds, workers,

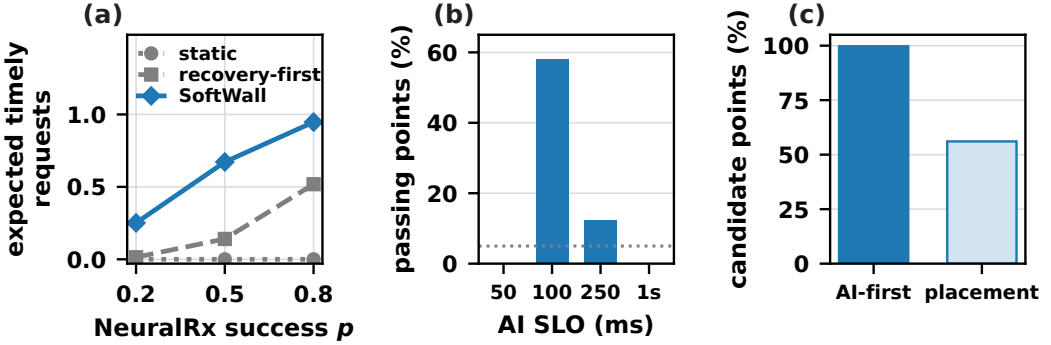


Fig. 7. Scheduling capacity and decision quality. (a) Expected timely requests at a representative four-cell, two-instance, one-GPU point with correlated outcomes. (b) Fraction of 513 mandatory-feasible trace points per SLO that pass the frozen 5% headroom screen. (c) Of 362 advanced points, a safe AI-first greedy is within 5% of the fixed-placement oracle at 361; flexible recovery placement has a positive upper-bound gain at 203.

and single-commit rule as SoftWall, but drains unresolved recovery before external AI. SoftWall adds AI-first placement only with a valid witness. The offline oracle sees the realized trace and supplies an unattainable upper bound.

Diagnostic Baselines. Debt-blind current-idle admission, local-only certificates, and uncontrolled MPS diagnose broken assumptions. They are not safe performance comparators and are not used to claim throughput gains.

Reproducibility. The anonymous artifact package contains source, protocols, manifests, analyzers, exact checker, independent verifier, result summaries, and figure builders. Hardware campaigns require proprietary Aerial/cuPHY and TensorRT components; model, checker, trace, and analysis paths do not. Node names and absolute paths are removed from the review artifact.

Use of generative AI. Generative-AI tools assisted with language editing, organization, bibliography normalization, and consistency-checking scripts. The authors designed the system and model, fixed protocols, executed experiments, inspected failures, and verified every claim against hashed artifacts. Generative AI produced no experimental measurement or ground-truth label.

6 Evaluation

We present scheduling outcomes before implementation qualification. Figure 7 answers the two headline questions: where debt-aware scheduling creates safe capacity, and which decision lever accounts for the remaining oracle gap. Later experiments establish that these schedules remain valid when mapped to CUDA workers, transport ownership, and failure handling.

6.1 RQ1: Debt awareness expands safe capacity

Conditional recovery creates a three-way scheduling tradeoff. Static reservation is safe but can strand capacity, debt-blind admission uses that capacity without protecting radio recovery, and SoftWall reclaims it only when a witness schedule exists.

Model Grid. The closed-form and exact multi-class capacity model covers 21,384 points over cells, cuPHY instances, GPUs, outcome probability and correlation, expiry, recovery bound, AI class, deadline, and offered load. Mandatory recovery is infeasible at 7,668 points. Among the 13,716 feasible points, SoftWall exceeds static reservation at 3,600 (26.2%), recovery-first trails its oracle

by at least 5% at 2,604 (19.0%), and debt-blind admission has positive violation probability at 4,338 (31.6%). Exhaustive order checking agrees with the compact capacity calculation in all 5,376 checked cases. The representative point in Figure 7a shows that conditional release increases safe capacity while static reservation remains at zero.

What the certificate prevents. E4 leaves two debts at 45 ms and exposes a context-256 request. Debt-blind admission permits a bound-respecting completion 12 ms beyond the 153 ms guard. E6b leaves one debt and makes context-64 unsafe at 89 ms by one millisecond. A prespecified diagnostic makes these allowed service vectors physical as a bound-padded diagnostic. Across two independent nodes, debt-blind E4 violates both guard and deadline in 40/40 selected rounds, and executing rejected E6b as a shadow violates the guard in 40/40. SoftWall rejects both, admits the adjacent 88 ms point, and records 0/120 selected safety violations (95% rule-of-three upper bound 2.5%). Jointly bound-valid counts are 39/39, 39/39, and 0/119, respectively.

Bound Sensitivity. The witnesses use the qualified $B_{\text{conv}} = 25$ ms contract. E6b disappears at or below 24 ms, while E4 remains until 19 ms. The observed recovery maximum is 13.465 ms, but a sample maximum cannot replace a service contract; lowering the bound defines another mode and requires requalification. The padded campaign is therefore a necessity diagnostic for the declared contract, not a production-rate or WCET claim.

Takeaway 1. Recovery debt is actionable scheduling state. It converts the unsafe capacity used by current-idle admission and the stranded capacity held by static reservation into capacity that can be admitted with an explicit all-fail witness.

6.2 RQ2: Decision quality and scheduling levers

Safe capacity is useful only if an online policy can place visible jobs well. We compare the candidate policy with exact fixed-placement schedules, an offline trace oracle, and a development-only flexible-placement upper bound.

Trace Headroom Screen. A frozen screen maps four BurstGPT calibration windows onto 2,592 topology, failure, recovery, SLO, and offered-unit points; 2,052 are mandatory-feasible. The pre-specified advance rule requires an oracle-normalized gap of at least 5% and a positive gap in every window. It advances 362 points: 298 at a 100 ms SLO and 64 at 250 ms, with none at 50 ms or 1 s. The 50 ms jobs rarely fit even with future knowledge, while recovery-first catches up at 1 s. Figure 7b therefore locates scheduling headroom in an intermediate deadline band.

Lever Decomposition. A certificate-preserving EDF/value AI-first greedy lies within 5% of the fixed-placement exact oracle at 361 of the 362 advanced points; the remaining point has a 7.02% gap. Flexible recovery placement has a positive upper-bound gain at 203 points: 0/102 one-GPU, 145/175 two-GPU, and 58/85 four-GPU points, with a 10.64% median positive gain. Selecting the frozen Sionna strata to minimize debt under zero or 0.02 absolute radio-value loss reduces no recovery debt. The result localizes remaining structural opportunity to cross-GPU recovery placement. Because this oracle excludes movement cost and physical qualification, it is a design target rather than an executed performance claim.

Original-Mode Negative Result. The calibration-only exact screen gives static global-safe, recovery-first, SoftWall, and the oracle the same radio decisions, arrivals, bounds, and one-Qwen-per-epoch capacity. After correcting a preserved absolute-deadline interpretation defect without excluding a request or opening the holdout, exact matching assigns 856 timely requests and 349,387 tokens to recovery-first, SoftWall, and the oracle. Static reservation exposes no qualified same-epoch Qwen slot and returns zero. Charging every realized recovery its full bound leaves recovery-first at 345,405 tokens, only 1.14% below SoftWall and below the prespecified 5% gate.

Table 4. Safety authority and AI-first scheduling utility are separate comparisons.

Policy	All-fail check	AI before debt clears	Finding
Debt-blind current-idle	No	Yes	40/40 E4 and 40/40 E6b guard violations
Static global-safe	Yes	Fixed residual only	0 timely requests in the original trace mode
Recovery-first	Yes	No	349,387 timely tokens
SoftWall	Yes	Only with witness	349,387 tokens; predicts safe boundary
Offline oracle	Realized trace	Earliest legal slot	349,387 tokens

Takeaway 2. A simple certificate-preserving AI-first order is sufficient for almost every fixed-placement point. The original physical mode contains no measurable ordering advantage, which rejects an executed throughput-superior optimizer claim. The remaining modeled opportunity is recovery placement across two or four GPUs; C175 supplies its upper bound, not an implemented result. We make no executed optimizer claim, and certificate-preserving recovery-first remains the strong safe baseline.

6.3 RQ3: Prediction accuracy and control cost

The scheduler must know before launch whether a candidate belongs to the safe-useful region. We test that classification against exact search, historical execution, and a boundary frozen before the physical outcome is opened.

Prediction Agreement. The envelope matches exact feasibility on 16,023 states and all 1,200 retrospective physical decisions. We then freeze seven boundary cases, their order, and node-specific seeds. Across two independent nodes and 180 rounds, model and runtime agree in every round: context-64 at 88 ms is admitted in 30/30, while 89 ms or later is rejected in 30/30. The campaign executes 720 actual NeuralRx requests, 330 recoveries, 90 Qwen units, and 720 commits without an observed deadline miss.

Decision Quality and Cost. On 600 heterogeneous small states, exact search accepts 504 and the deterministic candidate accepts 494. False-safe is 0/600 and false-conservative is ten; on the qualified mode, candidate and exact acceptance match all 16,023 states. A 2,800-state grid then spans 1–64 debts and capacities 1, 2, 4, and 8. Every returned certificate passes the verifier. At 64 debts, decision latency is 1.494 ms p99 and 2.197 ms maximum; verifier p99 is 0.131 ms.

Takeaway 3. The envelope predicts the safe-useful boundary before observing future CRC or realized service time, and the verified heuristic extends that decision beyond the small states handled by exact search.

6.4 RQ4: Accepted schedules survive physical execution

The preceding results operate on qualified service classes. RQ4 verifies that a complete accepted schedule executes the intended NeuralRx, shared recovery, external-AI, and radio-commit path.

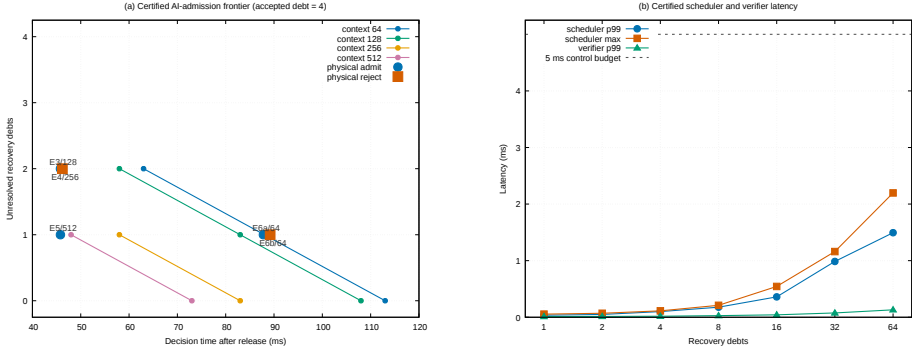


Fig. 8. Predictive scheduling and control cost. (a) The qualified AI-admission frontier moves with recovery debt, AI class, and decision time; prespecified physical cases lie on the predicted side. (b) Candidate construction and independent verification remain below the evaluated 5 ms control budget through 64 debts. Latencies are finite-run measurements, not WCETs.

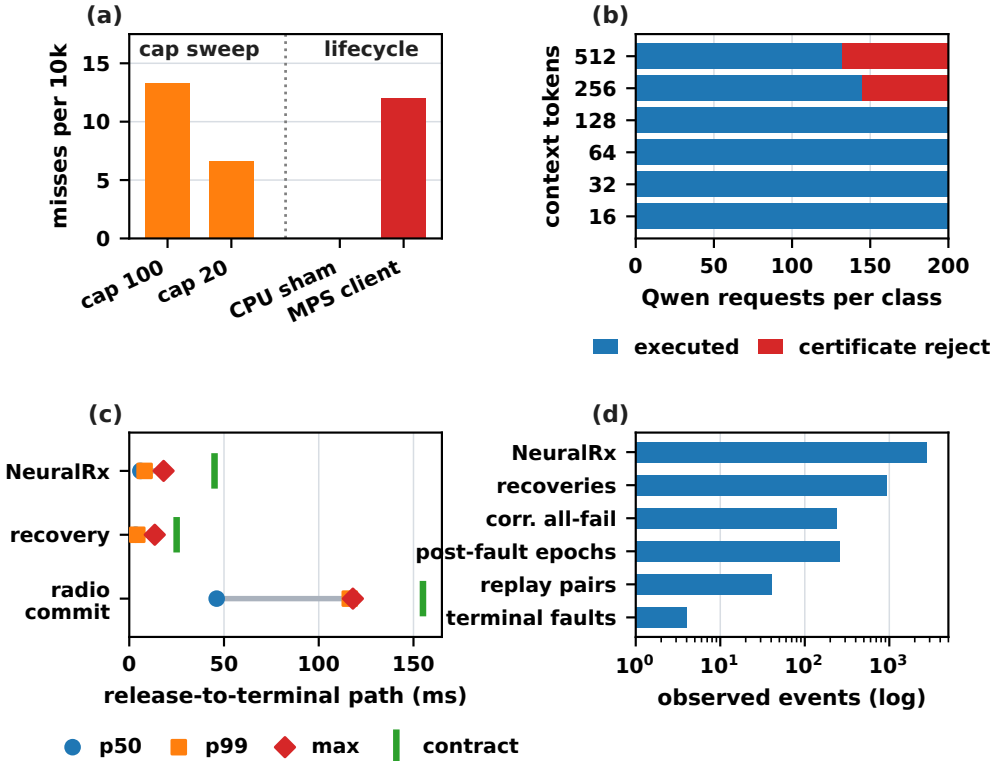


Fig. 9. Physical evidence for the scheduling contract. (a) MPS cap-only and lifecycle diagnostics retain deadline misses. (b) The scheduler admits every offered 16–128-token Qwen unit and rejects larger classes when recovery debt consumes the window. (c) Whole-path samples remain inside their mode contracts. (d) The fault campaign spans frequent execution and rare terminal events without an observed deadline or invariant violation.

Variable-Class Workload. The frozen P180/D155 campaign runs 600 epochs on each of two nodes with 16–512-token Qwen contexts. It executes 4,800 TensorRT NeuralRx requests, 1,312 shared-cuPHY recoveries, 1,077 Qwen units, and 4,800 single radio commits. The scheduler rejects 55 context-256 and 68 context-512 requests before launch and admits every offered 16–128-token request, as Figure 9b shows.

Timing and GPU Concurrency. Maximum NeuralRx, recovery, and radio-commit paths are 18.123, 13.465, and 118.022 ms; every executed Qwen unit meets its class bound and no kernel starts after its latest-start. Nsight verifies 191 NeuralRx–Qwen kernel-overlap segments totaling 8.932 ms and conventional–other-cell-NeuralRx overlap in 12/12 windows for 12.309 ms. These profiles establish physical co-execution, not service bounds.

Takeaway 4. The two-node campaign records no deadline, bound, guard, credit, or commit violation; 0/4,800 deadline misses gives a 95% rule-of-three upper bound of 0.0625%. This is finite-sample qualification of one warm synthetic mode.

6.5 RQ5: Faults preserve schedule ownership

Logical feasibility is insufficient when a stale message or lost reply can retire the wrong physical credit. We therefore combine finite-state checking with physical fault injection.

State and Physical Campaigns. A finite-state audit checks 51 combinations of epoch, generation, fence, and marker state without an invariant violation. The A0–A6 physical campaign covers clean operation, correlated NeuralRx failure, reply delay before and after completion, stale and duplicate responses, terminal channel fault, and post-fault continuation. Across qualified nodes it executes 700 epochs, 2,800 NeuralRx requests, 942 recoveries, and 2,800 commits with no observed deadline miss, including 40 replay pairs, four terminal faults, and 260 continuation epochs.

Fail-Closed Scope. One additional node violates the NeuralRx whole-path bound on its first round. We retain that run as UQ and do not pool it into the qualified aggregate. Fault containment therefore applies only to the two qualified nodes and declared fault/lifecycle classes.

Takeaway 5. Generation-safe ownership and physical fences preserve the schedule through the tested ambiguity classes. The 0/2,800 physical miss count has a 95% rule-of-three upper bound of 0.107%.

6.6 RQ6: MPS and lifecycle define the mode

MPS controls concurrency without certifying time, and worker lifecycle changes the service distribution on which a schedule relies. These mechanisms therefore belong to mode provenance rather than replacing the scheduler’s certificate.

MPS Diagnostics. A separate-client overrun produces 2/1,500 deadline misses at a 100% cap and 1/1,500 at a 20% cap. A matched lifecycle comparison observes 0/10,000 misses for a CPU socket sham and 12/10,000 for the GPU/MPS-client arm. Figure 9a reports the two campaigns separately; it does not imply a monotonic cap effect.

Lifecycle Envelope. The matrix contains five qualified or partial subsets and five UQ modes; the claim does not require or imply a 10/10 lifecycle matrix. Qualified evidence covers warm persistent execution, the first request after 30 s idle, MPS restart followed by requalification, radio continuity during Qwen reload, and same-worker reconnect. Optional-service availability during restart or reload is outside the claim. Process/model cold, 5 min and 30 min idle, GC-on, and worker process replacement remain UQ. A single-node replacement result is “Development canary only” because it lacks an independent holdout. Durable journal recovery would be a new mechanism and remains outside this paper.

Takeaway 6. Cap selection and current queue state do not authorize admission. Scheduling bounds remain valid only for the topology, placement, software, and lifecycle fingerprint that produced them.

6.7 RQ7: Deployment exit gates

The synthetic contract does not establish production timing. We use a vendor integration threshold to falsify candidate implementations while keeping target-DU timing as the first unresolved gate. **Production Timing.** Aerial testMAC expects CRC-bearing UL indication by $T_0 + 4.5$ ms. Under clean, warm, no-AI conditions, local wait-then-recover, same-GPU speculative, and precomputed cross-GPU paths complete 0/1,000 within 4.5 ms. Raw-IQ P2P moves channel estimation through CRC to GPU1 and reaches 852/1,000; a shared cuPHY/TensorRT stream reaches 885/1,000. The frozen persistent, stream-ordered fast path reaches 995/1,000 with 3.813 ms median and 3.995 ms p99, but five late completions reject the gate. The holdout stays closed. TestMAC is a controlled developer L2 rather than a field-DU d_{MAC} source.

Mechanism Attribution. A 300-request profile attributes the remaining tail to remote channel estimation: GPU p50/p99/max are 0.866/4.873/9.592 ms and correlation with pair wall time is 0.9603. P2P copies average 36.261/17.150 μ s, while derate matching exposes a 1,060.105 μ s host call despite 0.168 ms GPU p99. TensorRT enqueue averages 7.443 μ s and its graph has 0.886 ms GPU p99. These profiles are mechanism evidence rather than qualification or WCET.

Unsupported External TDL-A. External Aerial TDL-A remains unsupported. Across raw and normalized inputs, public-reference radio settings, one-antenna frequency/time channel execution, and a public-notebook-style FP32 engine, conventional decoding succeeds in every ten-TB arm and NeuralRx succeeds in none. Aerial TDL-A remains UQ.

Supported Sionna Holdout. A fixed development matrix selects Sionna CDL-D/E at 100 ns and retains A/B/C failures. A disjoint-seed holdout uses five Es/No strata and 50 blocks per stratum per model. Both pipelines pass 50/50 at 10 dB. CDL-D yields 153/250 conventional and 164/250 neural successes, with 16 neural-only and five conventional-only outcomes. CDL-E yields 155/250 and 163/250, with 15 neural-only and seven conventional-only outcomes.

Takeaway 7. The aggregate 31 neural-only versus 12 conventional-only outcomes give a two-sided exact $p = 0.00540$: optional NeuralRx has conditional radio value in the frozen Sionna CDL-D/E mode. That channel has not passed production timing or integrated service-vector qualification, while the best 4.5 ms testMAC candidate still misses 5/1,000. These failed gates bound the scheduler claim to qualified synthetic modes.

7 Related Work

Prior systems establish AI-and-RAN allocation, accelerator selection, GPU slack reclamation, and AI/conventional PHY choice. Table 5 compares the state protected by each scheduler.

YinYangRAN and CloudRIC schedule fractions or queued requests; Interplay, CAORA, HAF, and distributed AI-RAN cover orchestration, MIG, heterogeneous placement, and worker location [1, 6, 10–13]. Concordia reclaims vRAN CPU capacity, while DARIS, REEF, XSched, and nvtaskset control GPU stages, kernels, or contexts [2–5, 14]. Their scheduling units are resources or queued jobs, while recovery debt is future conventional work tied to one unresolved radio result.

ARCHES selects PHY experts, OCUDU exposes in-DU timing classes, and SMEC coordinates RAN/edge SLOs [8, 9, 16]. They rule out novelty claims for coexistence, fallback, or deadline scheduling. Their public designs do not expose a cross-instance all-fail recovery calendar atomically joined to an external-AI lease through physical completion.

Table 5. Closest-work distinction in public designs.

System	Primary protected/allocated object	State not exposed to its scheduler
YinYangRAN, CloudRIC, Concordia	Fraction, queue/deadline, or reserved capacity	Unqueued same-TB recovery debt
DARIS, REEF, XSched, nvtaskset	GPU stage, kernel, context, or partition	Radio recovery, transport ownership, and single commit as one certificate
ARCHES, OCUDU, SMEC	PHY choice, timing class, or edge SLO	Cross-instance all-fail debt plus atomic external-AI lease
SoftWall	Same-TB recovery obligation	Explicitly modeled, certified, and carried through physical completion

Scoped Difference. Across public designs reviewed through September 2026, we found no system combining an executable all-fail calendar for unresolved same-TB recovery, external-AI leasing through physical retirement, and a provenance-qualified predictive envelope. This is a scoped literature finding, not proof about unpublished work or other terminology.

8 Limitations

Timing Contract. P180/D155 is a synthetic harness contract: P is the inter-release period, not an NR slot duration, and D155 is not a production HARQ deadline. A production d_MAC trace builder and validator pass 24 unit tests, but a target DU/FAPI trace is unavailable. A new vector requires physical requalification; we make no production HARQ guarantee.

Finite-Sample Qualification. Zero observed violations establish neither WCET nor zero failure probability. Bounds belong to one hardware, software, placement, workload, and lifecycle fingerprint. All authoritative hardware results use A100s; NVLink P2P does not establish cross-family or inter-node data-plane validity.

Unresolved Long Tail. In C158 attempt 4, one NeuralRx completion reached 350.948 ms while the 400-request run’s p99 was 8.111 ms; the run had 4/400 deadline misses. Targeted idle, MPS restart and requalification, Qwen reload, and reconnect campaigns did not reproduce it. Its cause remains unresolved; GPU/driver hangs remain outside the fault model.

Workload Mapping. BurstGPT supplies real arrivals and token lengths; Qwen2.5-1.5B prefill is a representative external workload. Token buckets and AI deadlines are experimental mappings; other workloads need new bounds and replay qualification.

9 Conclusion

SoftWall represents optional-NeuralRx recovery debt, maintains a multi-cell all-fail certificate, and leases external AI through physical completion. Measured counterexamples derive its refinement chain, while the capacity model locates safe reclamation and identifies cross-GPU recovery placement as the main remaining headroom. Two-node evidence validates qualified synthetic modes; finite samples, unsupported channels, and absent target-DU timing establish neither WCET nor production HARQ.

References

[1] Ganesh Ananthanarayanan, Matthew Balkwill, Xenofon Foukas, Zhihua Lai, Bozidar Radunovic, Connor Settle, and Yongguang Zhang. 2025. Distributed AI Platform for the 6G RAN. In *ACM Workshop on Open and AI RAN*. ACM, New York, NY, USA, 15–21. <https://doi.org/10.1145/3737900.3770167>

[2] Amir Fakhim Babaei and Thidapat Chantem. 2025. DARIS: An Oversubscribed Spatio-Temporal Scheduler for Real-Time DNN Inference on GPUs. In *ACM/IEEE Design Automation Conference*. IEEE, Piscataway, NJ, USA, 1–7. <https://doi.org/10.1109/DAC63849.2025.11132423>

- [3] Joshua Bakita and James H. Anderson. 2025. Hardware Compute Partitioning on NVIDIA GPUs for Composable Systems. In *ECRTS*, Vol. 335. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, Wadern, Germany, 21:1–21:25. <https://doi.org/10.4230/LIPIcs.ECRTS.2025.21>
- [4] Xenofon Foukas and Bozidar Radunovic. 2021. Concordia: Teaching the 5G vRAN to Share Compute. In *ACM SIGCOMM*. ACM, New York, NY, USA, 580–596. <https://doi.org/10.1145/3452296.3472894>
- [5] Mingcong Han, Hanze Zhang, Rong Chen, and Haibo Chen. 2022. Microsecond-Scale Preemption for Concurrent GPU-Accelerated DNN Inferences. In *USENIX OSDI*. USENIX Association, Boston, MA, USA, 539–558. <https://www.usenix.org/conference/osdi22/presentation/han>
- [6] Haiyuan Li, Yulei Wu, and Dimitra Simeonidou. 2026. Deadline-Driven Hierarchical Agentic Resource Sharing for AI Services and RAN Functions in AI-RAN. arXiv:2605.07547 [cs.NI] <https://arxiv.org/abs/2605.07547>
- [7] NVIDIA Corporation. 2026. *Multi-Process Service Documentation*. NVIDIA. <https://docs.nvidia.com/deploy/mps/>
- [8] Timothy O'Shea, Matthew Pennybacker, and Andriy Kharchenko. 2026. The OCUDU dApp Platform: An Open Runtime and E3 Interface for Real-Time AI-RAN. arXiv:2609.07843 [cs.NI] <https://arxiv.org/abs/2609.07843>
- [9] Neagin Neasamoni Santhi, Davide Villa, Michele Polese, Salvatore D'Oro, Yunseong Lee, Koichiro Furueda, and Tommaso Melodia. 2026. ARCHES: Adaptive Real-Time Switching of AI Models for the RAN. arXiv:2604.23397 [cs.NI] <https://arxiv.org/abs/2604.23397>
- [10] Leonardo Lo Schiavo, Jose A. Ayala-Romero, Andres Garcia-Saavedra, Marco Fiore, and Xavier Costa-Perez. 2024. YinYangRAN: Resource Multiplexing in GPU-Accelerated Virtualized RANs. In *IEEE INFOCOM*. IEEE, Piscataway, NJ, USA, 721–730. <https://doi.org/10.1109/INFOCOM52122.2024.10621380>
- [11] Leonardo Lo Schiavo, Gines Garcia-Aviles, Andres Garcia-Saavedra, Marco Gramaglia, Marco Fiore, Albert Banchs, and Xavier Costa-Perez. 2024. CloudRIC: Open Radio Access Network (O-RAN) Virtualization with Shared Heterogeneous Computing. In *ACM MobiCom*. ACM, New York, NY, USA, 558–572. <https://doi.org/10.1145/3636534.3649381>
- [12] Syed Danial Ali Shah, Maryam Hafeez, Abdelaziz Salama, and Syed Ali Raza Zaidi. 2025. Proactive AI-and-RAN Workload Orchestration in O-RAN Architectures for 6G Networks. arXiv:2507.09124 [cs.NI] <https://arxiv.org/abs/2507.09124>
- [13] Syed Danial Ali Shah, Zeinab Nezami, Maryam Hafeez, and Syed Ali Raza Zaidi. 2025. The Interplay of AI-and-RAN: Dynamic Resource Allocation for Converged 6G Platform. In *IEEE INFOCOM Workshops*. IEEE, Piscataway, NJ, USA, 1–6. <https://doi.org/10.1109/INFOCOMWKSHPS65812.2025.11152939>
- [14] Weihang Shen, Mingcong Han, Jialong Liu, Rong Chen, and Haibo Chen. 2025. XSched: Preemptive Scheduling for Diverse XPU. In *USENIX OSDI*. USENIX Association, Boston, MA, USA, 671–692. <https://www.usenix.org/conference/osdi25/presentation/shen-weihang>
- [15] Yuxin Wang, Yuhan Chen, Zeyu Li, Xueze Kang, Yuchu Fang, Yehu Zhou, Yang Zheng, Zhenheng Tang, Xin He, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, and Xiaowen Chu. 2025. BurstGPT: A Real-World Workload Dataset to Optimize LLM Serving Systems. In *ACM SIGKDD*. ACM, New York, NY, USA, 5831–5841. <https://doi.org/10.1145/3711896.3737413>
- [16] Xiao Zhang and Daehyeok Kim. 2026. Enabling SLO-Aware 5G Multi-Access Edge Computing with SMEC. In *USENIX NSDI*. USENIX Association, Boston, MA, USA, 1759–1776. <https://www.usenix.org/conference/nsdi26/presentation/zhang-xiao>